

Prozessdokumentation: Website-Migration mit Claude Code

Projekt	KI Beratung Hessen (KIBH): WordPress/Elementor → statische Website
Bearbeiter	Hari
Projektverantwortlicher	Willy Li
Alt (WordPress)	https://kiberatunghessen.com
Neu (statisch)	https://kibh-webseite.vercel.app

Zahlenbasis. Alle Messwerte stammen aus dem Projekt selbst: Sitzungstranskripte (275 Einträge), `git log`, `PROGRESS.md`, `docs/arbeitsbericht.md`, `docs/performance-baseline.md` und zwei Lighthouse-Läufen vom 10.09.2026. Geschätzte Werte sind als solche gekennzeichnet.

1. Setup & Baseline

Umgebung

Betriebssystem	Windows 11 Pro. Grundsätzlich egal, Linux und macOS gehen genauso
Terminal	PowerShell
Editor	VS Code
Node.js	ab v22.0.0 (Mindestversion von Claude Code), hier v24.19.0
Claude Code	CLI im Terminal, nicht die Desktop-App. Version 2.1.267
Git / Hosting	Git + GitHub, Deployment auf Vercel (temporär)

- Der Bericht gilt für Website-Migration und kleine Verbesserungen.
- Zielbild: statische Website, höchstens minimale Backend-Teile wie die Terminbuchung für KIBH.

Claude Code CLI: Installation

Läuft schnell, wenn man schon Ahnung hat. Sonst Schritt für Schritt:

1. **Node.js prüfen:** Laufzeitumgebung, mit der JavaScript-Programme außerhalb eines Browsers laufen. Claude Code wird damit betrieben. `node --version` und `npm --version`. Kommen zwei Versionsnummern → installiert. Kommt ein Fehler, aktuelle Version von <https://nodejs.org/en/download> holen, installieren, danach nochmal prüfen. npm braucht keine eigene Prüfung, es kommt mit Node mit.
2. **Claude Code installieren und verifizieren:** `npm install -g @anthropic-ai/claude-code`, danach `claude --version`.
3. **Projektordner anlegen**, in dem die gesamte Entwicklung passiert.
4. **PowerShell in diesem Ordner öffnen**, dann `claude` starten.

5. **Einloggen** mit einem Account, der Claude Code unterstützt (Pro oder Max).

Authentifizierungs- oder Pfadprobleme bei der Einrichtung selbst: keine.

Voreinstellungen

- **CLAUDE.md anlegen** und Ziel, Aufgabe und Projektregeln hineinschreiben. Somit hat Claude vor jeder Session Kontext, ohne dass ich ihn erneut erkläre. Ein ausgefülltes Template dafür liegt bei mir bereit.
- **Skills installieren.**
 - **frontend-design** : liegt im offiziellen Marketplace, also über den eingebauten Befehl:

```
/plugin install frontend-design@claude-plugins-official
```

Zweck: drückt Claude weg vom generischen KI-Look („Inter font + purple gradient“). Gut zu haben, im Projekt aber kaum genutzt.

- **webapp-testing** : liegt nicht im Marketplace. Claude hat `SKILL.md`, `LICENSE.txt`, `scripts/with_server.py` und `examples/` aus `anthropics/skills` nach `~/.claude/skills/webapp-testing/` kopiert, also als User-Level-Skill, nicht projektlokal unter `.claude/skills`. Quelle: <https://github.com/anthropics/skills/tree/main/skills/webapp-testing> Zweck: Claude steuert einen echten Browser: aufrufen, scrollen, messen, Screenshots nach `tests/screenshots/`. Damit prüft er sein Ergebnis selbst, bevor er es mir vorlegt. Ohne diesen Skill kann Claude die Originalseite nur als Quelltext lesen, mit ihm kann er sie **ansehen**.
- Bilder nach `assets/img/` legen.
- Ordner als GitHub-Repository initialisieren.

Tech-Stack

- Statisches HTML/CSS/JS. Kein Framework, kein Bundler, null Abhängigkeiten.
- Node.js nur für ein 141-zeiliges Build-Skript (`src/` → `dist/`).
- Python nur für Hilfswerkzeuge: Playwright-Tests, WebP-Kodierung, Freistellen.

Warum das zum Prompting-Workflow passt

- **Nichts steht zwischen Anweisung und Ergebnis.** Was in `main.css` steht, ist genau das, was im Browser erscheint. Bei einem Framework kann eine Anweisung fachlich richtig sein und trotzdem nichts Sichtbares erzeugen, weil ein Build-Schritt, eine Konfiguration oder eine Komponentengrenze dazwischenliegt. Der Fehler ist dann schwer zuzuordnen: lag es am Prompt oder am Werkzeug?
- **Ein Begriff, ein Ort.** Eine Farbe ist ein Token in einer Datei. Claude muss nicht herausfinden, welche von fünf Schichten den Wert besitzt.
- **Keine Versionsdrift.** Framework-APIs ändern sich, und das Wissen eines Modells über eine bestimmte Version kann veraltet sein. HTML, CSS und JavaScript sind stabil.
- **Sofortige Sichtprüfung.** `webapp-testing` öffnet die gebaute Seite direkt. Kein Dev-Server, kein Hot Reload, kein Build-Fehler, der erst behoben werden muss. Die Schleife Auftrag → Ausführung → Sichtprüfung bleibt kurz, und genau von dieser Schleifenlänge hängt das Tempo ab.
- **Kleine, lesbare Diffs.** Ein missglückter Schritt ist ein Commit, den man löscht.
- **Für eine Migration besonders:** Quelle ist HTML, Ziel ist HTML. Es gibt keine Übersetzungsschicht, in der etwas verloren gehen kann.

Initialer Stand (visuelle Baseline)

- Vollständige Baseline in `docs/baseLine/` : alle 10 Seiten der alten WordPress-Site, jeweils Desktop (1440 px) und Handy (390 px), als Vollseite. Daneben dieselben Seiten des Neubaus als direkte Vorher/Nachher-Paare.
Dateischema: `<seite>__<viewport>__alt.jpg` und `__neu.jpg`.
- **Lehre für das nächste Projekt:** die Baseline gehört an den Anfang, bevor die erste Codezeile existiert, und Claude nimmt sie selbst auf. Es ist der einzige Moment, in dem sich das „Vorher“ festhalten lässt. Der Kunde kann die alte Seite jederzeit abschalten, danach ist der Vergleich weg.

Modelle, Effort, Usage, Modus

- Claude hat mehrere Modelle. Nicht immer das stärkste nehmen. Bild: Modelle sind Mitarbeiter, schwere Aufgaben gehen an die erfahrenen Kollegen.
 - **Sonnet:** kleine Fragen, Meinung einholen, Kleinkram.
 - **Opus:** Entwicklung und Planung, alles was Nachdenken braucht.
- **Effort-Stufe:** mindestens *High*. Für ernstere Sachen *xHigh*, für lange und wichtige Arbeit *Max*.
- **Kontingent:** mit Claude Pro (20 €/Monat), Opus auf Max Effort, ca. 2,5 bis 3 h Arbeit bis zum Limit, danach 5 h Wartezeit.
- **Modi:** für uns zwei relevant.
 - **Plan-Modus** beim Planen. Claude darf nichts ändern oder implementieren.
 - **Auto-Modus** bei der Umsetzung. Claude darf Dateien bearbeiten.

Session und Projekt

- Session beenden: `/exit`.
- `/compact` fasst den bisherigen Verlauf zusammen, d. h. am nächsten Tag geht es mit weniger Tokens weiter.
- Eine Session = ein Thema (normalerweise).
- Ein Projekt, also der angelegte Ordner, besteht aus mehreren Sessions.

Prompts mit KI-Hilfe schreiben

- Prompts für Claude Code mit einer zweiten KI vorbereiten (Claude Chat, ChatGPT).
 - Ablauf: Ich habe die Idee → gebe der KI allen Kontext und alle Informationen → lasse den Prompt schreiben → **jedes Wort selbst prüfen** → erst dann in Claude Code einfügen.
 - Ergebnis: deutlich genauere Aufträge und weniger Nacharbeit.
-

2. Migration & Core Build

Der Arbeitszyklus (gilt für A und B)

Auftrag → Ausführung → Sichtprüfung im Browser (erst Claude selbst, dann ich) → ggf. Korrektur. Und das in der Schleife.

- Klare Phasen wie „Struktur aufbauen → Design/CSS nachbauen → Inhalte befüllen“, wie bei klassischer Entwicklung mit echten Entwicklern, wurden hier nicht eingesetzt.
- Bei großen Aufträgen (erster Auftrag, neue Features): Prompt mit KI-Hilfe erzeugen. Bei kleinen Sachen (ein Fehler, eine Farbe): direkt tippen.
- Gemessen lief die Schleife rund 40-mal. Auf jeden großen Auftragsprompt kamen etwa drei kurze Korrekturanweisungen, 26 lange gegen 81 kurze. **Kein einziger großer Auftrag lief ohne Nachbesserung durch.**

A: Wie ich vorgegangen bin

#	Schritt	gemessen
1	Erster Auftrag: Claude liest <code>CLAUDE.md</code> , dann Planung	2,5 h
2	Bestandsaufnahme der alten Website durch Claude	1 h
3	1:1-Nachbau aller Seiten + erstes Deployment auf Vercel	20 min + 40 min
4	Neues Feature: Deep Reading Engine (DRE)	ca. 2 h
5	Korrektur	ca. 3 h
6	Seite-für-Seite-Vergleich mit dem Original + Korrektur	ca. 10 h
7	Auftrag „Design deutlich verbessern“, Fehlschlag, verworfen	1,5 h
8	Qualitätspässe: Mobile, Kleinigkeiten, Größe	ca. 6 h
9	Abgabe + erste Reklamationen	ca. 2,5 h
10	Letzte Assets und Abschluss	ca. 2,5 h
11	Prozessdokumentation Phase 3 geschrieben	0 h 33
12	Zweite Reklamationen	1 h 20
13	Mobile-Audit	0 h 33

Gesamt gemessen: 29 h 24 aktive Arbeitszeit über 17 Tage. Mit einer großzügigeren 45-Minuten-Pausengrenze gerechnet: 35 h 44.

Auffällig: der eigentliche Bau war der kürzeste Schritt. Der Abgleich mit dem Original war der längste und macht allein ein Drittel des Aufwands aus.

B: Wie ich das nächste Mal vorgehen würde (empfohlen)

- **Erster Auftrag:** Template für den perfekten ersten Prompt, das viele Phasen zusammenfasst. Liegt bei mir bereit, muss nur mit den Projektinformationen gefüllt werden.
- Korrektur, falls nötig.

- Neue Features und ggf. wieder Korrektur.
- Kundenreview und Korrektur-Batch.

Warum B schneller ist

- Der erste Prompt ist so genau, präzise und vollständig (ca. 700 Zeilen), dass er fast alle Rückfragen vorwegnimmt.
- Trotzdem muss danach alles geprüft, korrigiert, ergänzt und perfektioniert werden. Der erste Prompt spart die Anläufe, nicht die Sorgfalt.

Prompt-Bibliothek

- Alle Prompts sind in den Sitzungstranskripten gespeichert.
- Der erste Prompt, also das Template, ist der einzige, der sich wirklich wiederverwenden lässt.
- Eine detaillierte Zusammenfassung des Verlaufs steht in `docs/arbeitsbericht.md`.

Zwei Prompts, die tatsächlich versagt haben

- **„Bau die E-Rechnung-Seite nach“** (25.08.). Texte, Struktur und Farben stimmten, aber ohne Animationen und ohne die Hintergrund-Atmosphäre. Meine Rückmeldung: *„du hast nur die Texte kopiert und daraus eine rohe HTML-Datei gemacht.“*
- **„Setze den kompletten Design-Verbesserungsplan in einem Batch um“** (30.08.). Technisch sauber ausgeführt, sah aber schlechter aus als vorher. Komplette Verwerfung, ca. 1,5 h Totalverlust.

Do's & Don'ts

Was Claude sofort versteht

- Alles, was eindeutig und klar formuliert ist.
- Kleine, isolierte Aufgaben („diese eine Karte“, „dieser eine Button“).
- Wenn er zweifelt, soll er fragen. Deshalb an jedes Auftragsende: **„Ask if something is not clear.“**

Womit das Tool überfordert ist

- „Bau mir eine Website, mach keinen Fehler.“ Kein Kontext, keine Regeln, kein Maßstab. Genau dagegen existiert `CLAUDE.md`.

DO

- KI bei der Erstellung der Prompts mithelfen lassen.
- `CLAUDE.md` und den ersten Prompt sorgfältig schreiben. Beide bestimmen die Qualität aller folgenden Sessions.
- Claude committen lassen, pushen nur auf ausdrückliche Erlaubnis (siehe Abschnitt 3).
- Claude fragen, ob die Website den 8 Grundsätzen einer guten Website folgt: *point of view, typography that does work, a restrained color system, hierarchy that breathes, imagery with intent, motion that whispers, mobile that's designed not shrunk, the invisible expensive stuff.*
- Modus und Modell bewusst wählen.

DON'T

- Prompts unbedacht direkt in Claude Code tippen.
- Keine Ahnung davon haben, was man da eigentlich macht.

Refactoring vs. Scratch

- Alter Code wurde als Vorlage recycelt, aber „recycelt“ heißt hier nicht, dass WordPress- oder Elementor-Code übernommen wurde. Der Elementor-Output ist unbrauchbar: 1 363 KB gebündeltes CSS, von dem die Seite einen Bruchteil nutzt.
 - Recycelt wurden Inhalt und gemessene Design-Werte. Neu geschrieben wurde das gesamte Markup und CSS. Ergebnis: 126 KB statt 1 363 KB.
 - Neue Features wurden komplett neu generiert.
-

3. Revisionen & Workflow

Inhaltliche Anpassungen (Texte und Bilder tauschen)

- Deutlich schneller als im WordPress-Backend, und die Änderungen lassen sich als Batch übergeben. Claude tauscht nicht nur das Bild, sondern passt das Design gleich mit an.
- Gemessen an Willys zweitem Batch am 09.09.: neun Punkte auf einmal (zwei Textkorrekturen, zwei Rollen, ein neues Foto, eine fertiggestellte Karte, ein CSS-Bug site-weit, Navigation umgebaut, Logo-Größen, eine Sektion verkleinert) in **1 h 20 inklusive Tests und Sichtprüfung**. Die reinen Textkorrekturen darin: wenige Minuten.
- **Gegenrechnung WordPress:** ein einzelner Texttausch im Backend ist per Klick schneller als ein Prompt. Der Vorteil kippt, sobald es mehrere Änderungen auf einmal sind oder die Änderung Layout-Folgen hat. Dann macht Claude in einem Durchgang, wofür im Backend viele Klicks plus Nacharbeit am Theme nötig wären.

Visuelles Finetuning (CSS-Details)

Claude Code beherrscht das sehr gut:

- **Spacings:** „die Abstände nach den CTA-Bändern sind zu groß“: 156 → 116 px, 172 → 116, 216 → 159. Umgesetzt über eine Regel, die auf die *Form* der Sektion greift, nicht auf die zwei namentlich genannten Stellen.
- **Mobile Breakpoints:** Audit über 12 Seiten × 6 Handybreiten (320–430 px). Gefunden und behoben: 6 px horizontaler Seitenscroll auf `/kontakt/` bei 320 px, plus Wortumbrüche mitten im Wort auf sechs Seiten.
- **Hover-Effekte:** sechs Mikrointeraktionen, bewusst dezent. Eine musste nachgebessert werden: „*I can't see the cursor glow*“, zu schwach eingestellt und nur auf einer Seite ausgespielt.

Grenze: Claude kann jeden gewünschten Wert exakt umsetzen, aber nicht entscheiden, welcher Wert gut aussieht. Das musste jedes Mal ich beurteilen.

Versionskontrolle (Git)

- **Commits macht Claude selbst**, nach jeder abgeschlossenen logischen Änderung, nicht nach jedem einzelnen Schritt. Gemessen: 86 Commits.
- **Push auf GitHub nur mit meiner ausdrücklichen Erlaubnis.** Grund: solange nichts gepusht ist, reicht es, einen Commit lokal zu löschen, wenn etwas schiefgeht. Im Projekt war das zweimal wertvoll, einmal beim verworfenen Design-Versuch und einmal, als eigenes Arbeitsmaterial fast in einem öffentlichen Repository gelandet wäre.
- **Kontrolle behalten:** ein Commit pro logischer Änderung, aussagekräftige Commit-Messages, und `PROGRESS.md` als datiertes Protokoll mit Commit-Hashes.

4. Bottlenecks & Grenzen

Ungeschminkt. Das ist der Teil, der die Fallstudie belastbar macht.

Context Window & Vergesslichkeit

- **11 Kompaktierungen im Projekt.** Jede längere Sitzung lief voll, danach hatte Claude nur noch eine Zusammenfassung statt des echten Verlaufs. Deswegen wurde `PROGRESS.md` eingeführt, in der nach jeder Session ein kurzes Update steht.
- **Hat Claude bestehende Styles überschrieben?** Nicht durch Vergesslichkeit, aber durch zu breite Regeln: eine Regel, die für eine einzelne Seite gedacht war, griff auf ein geteiltes Element und veränderte damit alle Seiten. So etwas fällt bei der Sichtprüfung auf, nicht beim Lesen des Codes.

Bugs, Sackgassen, manuelle Eingriffe

Die Fehler fielen in zwei Klassen, und keine davon war ein Kreisen der KI:

- **Methodische Fehlannahmen.** Der größte Zeitverlust entstand, weil die Vorlage aus dem Quelltext statt aus dem gerenderten Bild rekonstruiert wurde. Animationen und Hintergründe gingen dabei verloren. Das kostete den kompletten Neubau einer Seite und änderte danach die Arbeitsweise für den Rest des Projekts.
- **Plattformbedingte Eigenheiten.** Windows-Zeileneenden, Groß- und Kleinschreibung in Dateinamen, Python nicht im Standardpfad. Jede dieser Fallen kostete einmal Zeit und ist danach dokumentiert, damit sie nicht wiederkehrt.

Dazu zwei Beobachtungen:

- **Endlosschleifen gab es nicht.** Wo Claude nicht weiterkam, hat er gefragt statt zu raten. Bei allem, was das Aussehen oder die Außenwirkung veränderte, wurde grundsätzlich gefragt.
- **Manuell eingreifen musste ich bei zwei Dingen:** Geschmacksentscheidungen und Fehlern, die nur auf einem echten Gerät auftraten.

Token- und Kostenaufwand

- **Abo:** Claude Pro, 20 €/Monat. Kein zusätzlicher API-Verbrauch.
- **Teuerste Posten**, nach Gefühl, da keine Token-Messung verfügbar ist:
 1. Seite-für-Seite-Vergleich (ca. 10 h). Viele Playwright-Läufe und viele Screenshots, und jeder Vergleich zieht Bilddaten in den Kontext.
 2. Der Neubau nach dem 1:1-Irrtum, also dieselbe Seite zweimal gebaut.
 3. Kontextverluste. Nach jeder Kompaktierung muss `PROGRESS.md` neu gelesen werden.
- **Billigster Posten:** die eigentliche Code-Erzeugung.
- **Einordnung:** bei 29 h 24 aktiver Arbeit liegen die reinen Werkzeugkosten des Projekts unter 20 €, denn das Abo lief ohnehin. Der Aufwand steckt in der Arbeitszeit, nicht in der Lizenz.
- **Grenze der Aussage:** Claude Code zeigt keinen Token-Zähler pro Aufgabe. Alle Angaben hier sind Beobachtung, keine Messung.

5. Fazit & B2B-Vergleich

Vergleichstabelle WordPress vs. Claude Code

Kriterium	WordPress (alt)	Claude Code (neu)
Lighthouse Performance (Handy / Desktop)	62 / 90	91 / 97
Lighthouse Accessibility	92	96
Lighthouse Best Practices / SEO	100 / 100	100 / 100
Largest Contentful Paint (Handy / Desktop)	6,9 s / 1,2 s	2,8 s / 1,0 s
First Contentful Paint (Handy / Desktop)	5,3 s / 1,0 s	2,6 s / 1,0 s
Cumulative Layout Shift (Handy / Desktop)	0 / 0,142	0 / 0,001
Total Blocking Time (Handy / Desktop)	140 ms / 20 ms	60 ms / 0 ms
Seitengewicht (Handy / Desktop)	1 184 / 875 KiB	256 / 256 KiB
Server-Antwortzeit	160 ms	10 ms
Main-Thread-Arbeit (Desktop)	3,5 s	0,4 s
JavaScript-Ausführung (Desktop)	0,5 s	0,0 s
CSS-Auslieferung	1 363 KB gebündelt	126 KB
Seiten mit seitlichem Scroll am Handy	4 von 10	0 von 10
Setup bis erste Live-Seite	ca. 2–5 h (Schätzung)	ca. 3 h (gemessen)
Gesamtaufwand Migration	nicht gemessen	29 h 24
Flexibilität bei Designwünschen	an Theme und Page-Builder gebunden	direkt im Quelltext
Wartung und Sicherheit	laufende Updates, Plugins als Angriffsfläche	keine Abhängigkeiten, kein PHP, keine Datenbank
Rückabwicklung von Fehlern	Backup oder Staging nötig	einen Commit löschen
Automatisierte Qualitätsprüfung	keine	11 Seiten × 5 Breiten je Lauf
Redaktion durch den Kunden	GUI im Backend, jeder kann Texte ändern	nur mit Entwickler

Messgrundlage Lighthouse: Version 13.4.1, 10.09.2026. Zwei Läufe je Seite, `--preset=desktop` und der Mobile-Standard mit gedrosselter CPU und gedrosseltem Netzwerk, so wie Google es für die Suchbewertung ansetzt.

Ehrliche Einordnung der Zahlen

- **Am Desktop ist WordPress nicht katastrophal:** 90 Performance, 100 bei Best Practices und SEO. Wer das Gegenteil behauptet, verkauft. Der Unterschied liegt dort bei Stabilität und Leerlauf: CLS 0,142 → 0,001, Main-Thread 3,5 s → 0,4 s, JavaScript 0,5 s → 0,0 s. Der erste Bildaufbau ist identisch, 1,0 s bei beiden.
- **Am Handy fällt die Entscheidung:** 62 gegen 91 Punkte, LCP 6,9 s gegen 2,8 s. Das ist der Wert, den Google am stärksten gewichtet, und 6,9 s liegt weit jenseits der 2,5-Sekunden-Schwelle für „gut“. Auf dem Gerät, mit dem die meisten Besucher kommen, ist der Unterschied also kein Feinschliff.
- **CLS ist ein Core Web Vital.** Am Desktop 0,142, also über der 0,1-Schwelle, gegen praktisch null. Am Handy sind beide bei 0.
- **Vier von zehn Seiten des Originals lassen sich am Handy seitlich schieben,** gemessen bei 390 px Viewport: Kommunikation 476 px, Kontakt 421, Automatische Rechnungsprüfung 411, Startseite 400. Alle zehn Seiten des Neubaus liegen exakt bei 390 px. Belege als Screenshot-Paare in [docs/baseline/](#).
- **Zwei Werte sind nicht besser:** der Speed Index am Desktop liegt bei 1,2 s gegen 1,1 s des Originals, praktisch gleich. Und die Accessibility-Differenz von 92 auf 96 ist eine Verbesserung, aber keine, die ein Kunde spürt. Gehört trotzdem genannt.

Hosting-Kosten (IONOS, Stand 10.09.2026)

	WordPress	Statische Seite
Tarif	WordPress Hosting Start	Webhosting Standard
Aktionspreis	3 €/Monat, 6 Monate	3 €/Monat, 6 Monate
Regulär danach	5 €/Monat	6 €/Monat
Speicher	25 GB	100 GB NVMe
Domain / Postfach	1 / 1	1 / 1

Für die statische Seite ist kein WordPress-Tarif nötig, da es weder Datenbank noch CMS gibt.

- **Beim reinen Hosting-Preis gibt es keinen Vorteil.** Der passende Nicht-WordPress-Tarif ist regulär sogar 1 €/Monat teurer. Er bringt dafür die vierfache Speichermenge, die eine statische Seite nicht braucht.
- **Claude Pro gehört nicht in diese Rechnung.** Das ist ein allgemeines Entwickler-Abo, kein Hosting-Posten pro Website. Es läuft unabhängig davon, wie viele Kundenprojekte darauf gebaut werden.
- **Wo der Kostenvorteil tatsächlich liegt:**
 1. Setup-Zeit.
 2. Keine kostenpflichtigen Premium-Themes oder -Plugins.
 3. Laufende Wartung: keine Update-Zyklen, keine Plugin-Konflikte, keine Kompatibilitätstests nach jedem WordPress-Release.
 4. Statisches Hosting ist auch kostenlos zu haben. Dieses Projekt läuft auf Vercels kostenloser Stufe. Die IONOS-Tabelle ist der angefragte Vergleich, nicht das, was hier tatsächlich bezahlt wird.

Persönliches Fazit

- **Ja, ich würde Claude Code für ein Kundenprojekt wieder wählen.**
- **Gründe:** schneller und einfacher zu benutzen als WordPress, flexibler bei Änderungen.
- **Aber:** wir haben weniger Kontrolle als bei WordPress.
- **Für welchen Kundentyp:** heute eignet sich Claude Code aus meiner Sicht für alle Kundentypen. Entscheidend ist der Mensch dahinter, er muss den Tech-Stack verstehen.

- Kunde mit rein statischer Website wie KIBH: Basis-Kenntnisse in Webentwicklung reichen.
- Website mit mehr Backend: entsprechend mehr Verständnis nötig.
- Hier war die Aufgabe eine Web-Migration, es wurde also viel Bestehendes wiederverwendet.

Weitere Vorteile

- **Kleinere Angriffsfläche.** Kein PHP, keine Datenbank, keine Plugins. Die häufigste WordPress-Einbruchsursache, ein veraltetes Plugin, existiert nicht.
- **Kein Anbieter-Lock-in.** Die Seite ist ein Ordner mit HTML, CSS und Bildern. Sie läuft auf jedem Webspaces, jedem CDN, jedem Objektspeicher. Ein Umzug ist ein Kopiervorgang.
- **Messbare Kernwerte statt Gefühl.** Die Lighthouse-Zahlen sind reproduzierbar und lassen sich einem Kunden vorlegen.
- **Migration deckt Altlasten auf.** Beim Nachbau fielen drei echte Defekte der Live-Seite auf, die dem Kunden niemand gemeldet hatte: 404 auf allen Unterseiten-Permalinks, kaputte Logo-Bilder, eine unsichtbare Überschrift. Das ist ein Verkaufsargument für sich.
- **Prüfbarkeit als Liefergegenstand.** Das Testskript kann bei jeder späteren Änderung erneut laufen. Bei WordPress gibt es diese Zusicherung nicht.

Weitere Nachteile

- **Der Kunde kann Inhalte nicht selbst pflegen.** Kein Backend, keine GUI. Jede Textänderung braucht jemanden mit Claude Code. Das ist der größte Nachteil gegenüber WordPress und gehört ins Angebot, entweder als Wartungsvertrag oder mit einem späteren schlanken CMS.
- **Die Qualität hängt an einem Menschen, der hinsieht.** Praktisch jede Korrektur kam daher, dass ich die Seite selbst im Browser beurteilt habe. Zwei Fehler waren ausschließlich so auffindbar, darunter einer, der nur auf einem echten Telefon auftrat. Ohne diese Prüfschleife wäre die Seite fertig und falsch gewesen.
- **Prompt-Qualität ist Projektqualität.** Präzise Aufträge lieferten brauchbare Ergebnisse, vage erzeugten Nacharbeit. Diese Fähigkeit muss man haben oder einkaufen.
- **Nicht belegt durch dieses Projekt:** das Verhalten bei umfangreicher Serverlogik, bei mehreren hundert Seiten oder bei Redaktionsteams.

Quellen und Nachweise

Nachweis	Ort
Detaillierter Arbeitsbericht, 13 Arbeitsschritte	<code>docs/arbeitsbericht.md</code>
Datiertes Sitzungsprotokoll mit Commit-Hashes	<code>PROGRESS.md</code>
Aufgabenstellung und Projektregeln	<code>CLAUDE.md</code>
Visuelle Baseline alt/neu, 10 Seiten × 2 Breiten	<code>docs/baseline/</code>
Messwerte Seitengewicht, Methodik, Vorbehalte	<code>docs/performance-baseline.md</code>
Design-Werte mit Fundstelle im Original	<code>docs/design-system.md</code>
URL-Zuordnung alt → neu, inkl. 404-Befund	<code>docs/url-map.md</code>
Commit-Historie	<code>git log</code>
Lighthouse-Rohdaten	<code>npx lighthouse <url> --preset=desktop</code>
IONOS-Preise	ionos.de/hosting/webhosting · ionos.de/hosting/wordpress-hosting